

A Fused Pallas Kernel for the Mixture-of-Experts MLP

Hyeonseok Jung

1 Problem statement

The expert MLP of a mixture-of-experts layer computes

$$y[i] = \text{down}[g(i)]^\top (\text{swish}(x[i] \text{ gate}[g(i)]) \odot (x[i] \text{ up}[g(i)])), \quad (1)$$

where x is (m, d) , up and gate are (e, d, f) , down is (e, f, d) , and $g(i)$ is the expert assigned to token i . Rows arrive sorted by expert, so the assignment is given by a vector of e group sizes summing to m . The router, the sort, the scatter back to token order, and biases are outside the scope of this report. Written as three `jax.lax.ragged_dot` calls, this lands two (m, f) intermediates in HBM and reads them back, so $6mf$ elements of intermediate activations are communicated through memory between the three matmuls. I fuse the up, gate, and down projections into a single Pallas kernel that keeps those intermediates in VMEM, removing that communication entirely, and measure it against the reference on one TPU v5e core under JAX 0.6.2, in bfloat16.

2 The fused kernel

The kernel evaluates

$$y[m, j] = \sum_i \left((\sum_k x[m, k] \text{ up}_e[k, i]) \odot \text{swish}(\sum_k x[m, k] \text{ gate}_e[k, i]) \right) \text{down}_e[i, j] \quad (2)$$

over a four-dimensional `pltpu.emit_pipeline` grid (m, i, j, k) , where m indexes tiles of rows, i the hidden dimension f , j the output dimension d , and k the contracted input dimension d , with block sizes (b_m, b_i, b_j, b_k) . Write $I = f/b_i$, $J = d/b_j$, and $K = d/b_k$ for the number of steps along each axis, and e for the expert owning row tile m .

Kernel 1 Fused expert MLP, one grid step per innermost iteration.

```
1: for  $m \leftarrow 0$  to row tiles  $- 1$  do
2:   for  $i \leftarrow 0$  to  $I - 1$  do
3:     for  $j \leftarrow 0$  to  $J - 1$  do
4:       for  $k \leftarrow 0$  to  $K - 1$  do
5:         if  $i = 0$  and  $j = 0$  and  $k = 0$  then
6:            $\text{acc} \leftarrow 0$   $\triangleright (b_m, d)$  output accumulator
7:         if  $j = 0$  and  $k = 0$  then
8:            $a \leftarrow 0, g \leftarrow 0$   $\triangleright (b_m, b_i)$  hidden accumulators
9:         if  $j = 0$  then
10:           $a \leftarrow a + x[m, k] \cdot \text{up}_e[k, i]$ 
11:           $g \leftarrow g + x[m, k] \cdot \text{gate}_e[k, i]$ 
12:         if  $k = K - 1$  then
13:           $\text{acc}[:, j] \leftarrow \text{acc}[:, j] + (a \odot \text{swish}(g)) \cdot \text{down}_e[i, j]$ 
14:         if  $i = I - 1$  and  $k = K - 1$  then
15:           $\text{out}[m, j] \leftarrow \text{acc}[:, j]$ 
```

Two float32 accumulators of shape (b_m, b_i) hold the up and gate partial sums for the current (m, i) pair, and one of shape (b_m, d) holds the output partial sums for the row tile. The combination $a \odot \text{swish}(g)$ is formed on the accumulator tile and fed straight into the down matmul, so neither it nor either factor

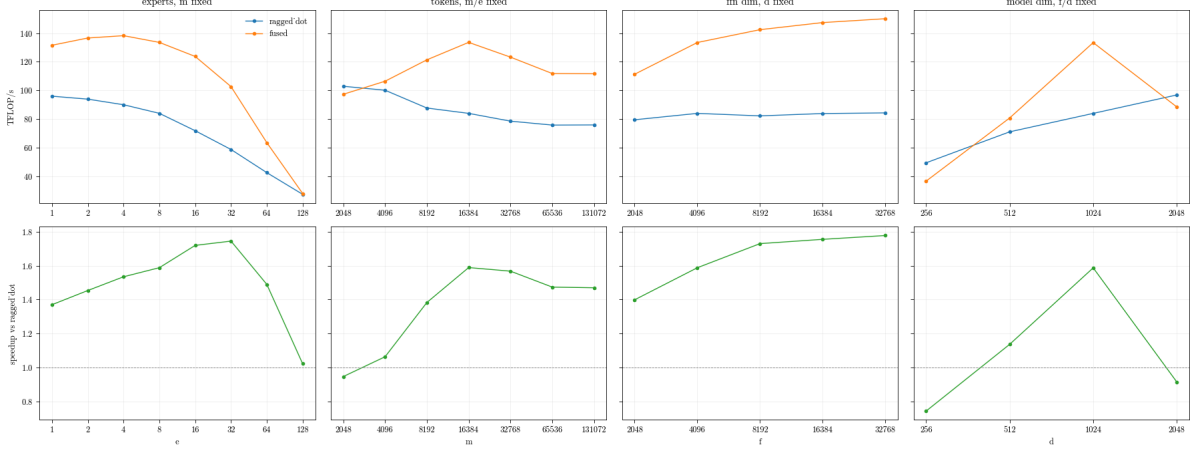


Figure 1: Shape sweeps: (a) expert count, (b) token count, (c) hidden dimension, (d) model dimension. Top row, achieved rate; bottom row, their ratio. One parameter doubles per point, the rest held at the reference shape $(m, d, f, e) = (16384, 1024, 4096, 8)$. In (b) each expert keeps 2048 rows; in (d) f/d stays at 4.

ever reaches HBM. The kernel accumulates in float32 throughout, where the reference rounds each ragged dot’s output to bfloat16.

The ragged structure enters through the block index maps, which return the expert of the current row tile from a small SMEM array. For one expert to own each tile, group boundaries must fall on multiples of b_m , so I pad x with zero rows before the call and gather the rows back afterwards; the row-tile count $\sum_j \lceil s_j / b_m \rceil$ is data-dependent and is passed in as an operand. Both the padding and the gather back are timed as part of the kernel in every measurement below.

3 Results

Every measurement uses the same block-size rule, derived from the shape and never tuned per point: $b_m = \min(256, m/2e)$, $b_i = \min(1024, f/4)$, $b_j = \min(1024, d)$, and $b_k = \min(512, d/2)$. Reported rates are $6mdf$ divided by the measured time, and the kernel’s output matches the reference to a relative and absolute tolerance of 2×10^{-2} at every point below. At the reference shape $(m, d, f, e) = (16384, 1024, 4096, 8)$ the fused kernel is $1.59\times$ faster, 3.09ms against 4.90ms. The removed traffic accounts for all of that: the reference’s elementwise stage does no matmul work and streams $3mf$ elements in 946 μ s, so the $6mf$ elements the fusion eliminates are worth 1.89ms at the same rate, against a measured saving of 1.82ms.

3.1 Shape sweeps

The expert count sweep (Figure 1a) shows a rise and then a collapse. The speedup improves from 1.37 at one expert to 1.75 at 32 experts, then falls to 1.03 at 128. The token count is fixed, so the mean group size is m/e and it shrinks as experts are added. Each expert’s three weight slabs are read to serve only the rows of its own group, which makes the arithmetic intensity with respect to weight traffic equal to the group size in FLOPs per byte. Once that number is small, both implementations spend their time streaming weights and their rates converge. The fusion removes communication between operations. It cannot remove traffic that the operation itself requires.

The token count sweep (Figure 1b) holds the group size at 2048 rows and grows the expert count with the token count. The speedup rises from 0.95 at $m = 2048$ to 1.59 at $m = 16384$, then settles near 1.47. The loss at the smallest point is a pipeline that never fills. One expert and 2048 rows give only 8 row tiles, so the prologue and the epilogue are a large fraction of the run.

The hidden dimension sweep (Figure 1c) is the regime the fusion targets. With d fixed at 1024, the speedup rises monotonically from 1.40 at $f = 2048$ to 1.78 at $f = 32768$. The reference holds a flat rate across the whole range, because its intermediate traffic and its FLOPs both grow in proportion to f and the ratio between them does not change. The fused kernel does not pay that traffic, so its rate climbs instead.

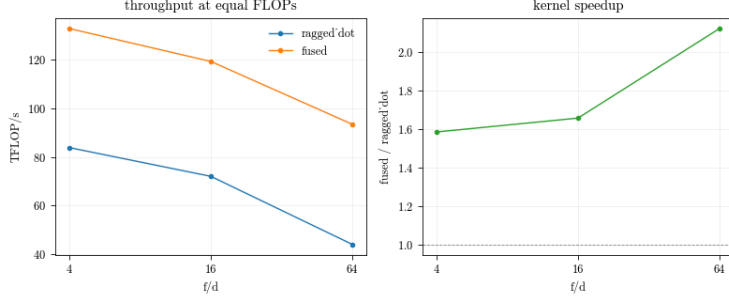


Figure 2: Constant FLOP count: (a) achieved throughput, (b) speedup. All three points are $m = 16384$, $e = 8$, $df = 2^{22}$.

The model dimension sweep (Figure 1d) is the weakest result. The speedup peaks at 1.59 at $d = 1024$ and falls away on both sides, to 0.75 at $d = 256$ and 0.91 at $d = 2048$. Both ends are failures of the block-size rule rather than of the fusion. At $d = 256$ the rule yields tiles that are too small for the MXU. At $d = 2048$ it gives the first shape in any sweep with $b_j < d$, which makes the kernel run the full k loop once per j block while only the last step does work.

3.2 Aspect ratio at equal FLOPs

The f sweep raises f/d and the total work together, so it cannot separate the two. Holding df fixed does: the three points in Figure 2 share m , e , and FLOP count, and differ only in the shape of the MLP. Only three such points satisfy $f > d$ with b_k a multiple of 128.

f/d	(d, f)	ragged_dot	fused	speedup	intermediates
4	(1024, 4096)	83.7	132.8	1.59	805 MB
16	(512, 8192)	72.1	119.2	1.66	1.61 GB
64	(256, 16384)	43.9	93.3	2.12	3.22 GB

Table 1: The equal-FLOPs comparison. Rates are TFLOP/s; the last column is the intermediate traffic the reference moves and the fused kernel does not.

The speedup rises from 1.59 to 2.12, and the mechanism is the last column of Table 1. At a fixed df product the intermediate volume mf grows as the square root of the aspect ratio, so each factor of four in f/d doubles the traffic the reference must move while its FLOPs and weight traffic stay constant. The fused kernel slows as well, since at $d = 256$ its matmuls are much narrower in the contracted dimension, but by less than half as much.

3.3 Routing skew

Everything so far uses balanced experts, the one routing distribution a real MoE never produces. Figure 3 scales the router logits by a skew parameter, so larger values concentrate tokens on fewer experts.

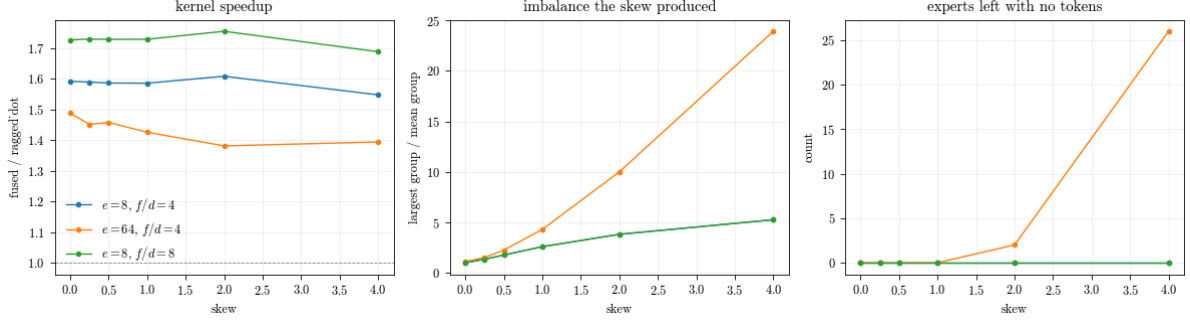


Figure 3: Effect of routing skew: (a) speedup, (b) the imbalance the skew produced, (c) experts left with no tokens. All three are $m = 16384$, $d = 1024$. The two $e = 8$ configurations share a group-size distribution, so their lines coincide in (b) and (c).

At eight experts the speedup barely moves, from 1.59 to 1.55 at $f/d = 4$, while the largest group grows to 5.3 times the mean. At 64 experts it degrades from 1.49 to 1.39 over a range where the largest group reaches 24 times the mean and 26 experts end up empty.

The difference is the padding tax. A group of s rows costs $\lceil s/b_m \rceil$ tiles, so the waste is at most $b_m - 1$ rows per non-empty group and empty groups cost nothing. At eight experts the mean group is 2048 rows against $b_m = 256$, so the waste is negligible; at 64 experts it is 256 rows against $b_m = 128$, and skew drives many groups below a full tile. What matters is the ratio of group size to tile height, not the imbalance itself.

4 Conclusion

The fused kernel beats three `ragged_dot` calls whenever the intermediates it removes are large next to the traffic that remains, and whenever there is enough work to keep its pipeline full. Over the shape sweeps it is faster at all but three of the 24 points, and the largest speedup measured anywhere is 2.12.

The favourable condition is a wide MLP. At equal FLOPs the speedup rises from 1.59 at $f/d = 4$ to 2.12 at $f/d = 64$ (Table 1), because the intermediate volume the reference moves grows with the aspect ratio while its FLOPs and its weight traffic do not. At a fixed $d = 1024$ the speedup rises with f to 1.78. Every shape I measured with $f > d$ and a block size matched to it is faster than the reference.

Two conditions limit the result. The first is the number of rows per expert. Expert weights are streamed by either implementation and are amortised over one group only, so once the mean group size approaches the tile height both become weight-bound. At $e = 128$ the mean group is 128 rows and the speedup is 1.03. Routing skew acts through the same channel, costing almost nothing at 8 experts and reducing the speedup from 1.49 to 1.39 at 64. The second condition is total work. At $m = 2048$ with a single expert there are only 8 row tiles, too few to fill the pipeline, and the kernel is 5% slower than the reference. The two remaining losses, at $d = 256$ and $d = 2048$, come from the single block-size rule rather than from the fusion, and per-shape tuning of (b_m, b_i, b_j, b_k) is the first thing to try there.